

Smart Hospital Patient Registration System

A multi-specialty hospital wants to digitize its outpatient registration system.

Every patient arriving at the hospital receives a registration number. The hospital management wants a Java application that maintains patient records efficiently while preventing duplicate registrations.

Each patient record consists of

☐ Patient ID

☐ Patient Name

☐ Mobile Number

☐ Department

☐ Doctor Name

The system should utilize Java Collection Framework and String manipulation techniques for storing, validating, searching, and displaying patient information.

Functional Requirements

Develop a Java program using ArrayList and HashMap to perform the following operations.

Menu

1. Register Patient
2. Search Patient
3. Display All Patients
4. Count Department-wise Patients
5. Exit

Requirement 1 – Register Patient (4 Marks)

Accept

Patient ID

Patient Name

Mobile Number

Department

Doctor Name

Perform validations.

☐ Patient ID should be unique.

☐ Patient Name should contain only alphabets.

☐ Mobile Number should contain exactly 10 digits.

☐ Department should automatically convert to uppercase.

☐ Remove unnecessary spaces from all string inputs.

Store the patient in

☐ ArrayList

☐ HashMap (PatientID as Key)

Requirement 2 – Search Patient (4 Marks)

Accept Patient ID.

Display complete patient details.

If not found,

Patient Not Found

Requirement 3 – Display All Patients (4 Marks)

Display in the following format.

ID Name Mobile Department Doctor

P101 Rahul 9876543210 CARDIOLOGY Dr.Arun

P102 Priya 9876543211 ORTHO Dr.Meena

Requirement 4 – Department-wise Count (4 Marks)

Count the number of patients in each department.

Example

CARDIOLOGY : 5

ORTHO : 3

NEURO : 4

Use

HashMap<String,Integer>

Requirement 5 – String Operations (4 Marks)

For every patient,

Perform

☐ Convert Name to Proper Case

☐ Remove leading/trailing spaces

☐ Display Name Length

☐ Display Department in Uppercase

Example

Input

raHUI kumAr

Output

```

1 1st java.util.*;
2
3 3ic class Main {
4
5 5:static ArrayList<Patient> patients = new ArrayList<>();
6 6:static HashMap<String, Patient> patientMap = new HashMap<>();
7
8 8:public static void main(String[] args) {
9
10 Scanner sc = new Scanner(System.in);
11
12 while (sc.hasNextLine()) {
13
14 String input = sc.nextLine().trim();
15
16 if (input.isEmpty()) {
17     continue;
18 }
19
20 int choice = Integer.parseInt(input);
21
22 switch (choice) {
23
24     case 1:
25
26         if (!sc.hasNextLine()) return;
27         String id = sc.nextLine().trim();
28
29         if (!sc.hasNextLine()) return;
30         String name = sc.nextLine().trim();
31
32         if (!validName(name)) {
33             break;
34         }
35
36         name = properCase(name);
37
38         if (!sc.hasNextLine()) return;
39         String mobile = sc.nextLine().trim();
40
41         if (!validMobile(mobile)) {
42             break;
43         }
44
45         if (!sc.hasNextLine()) return;
46         String department =
47             sc.nextLine().trim().toUpperCase();
48
49         if (!sc.hasNextLine()) return;
50         String doctor = sc.nextLine().trim();
51
52         if (patientMap.containsKey(id)) {
53             System.out.println("Patient ID already exists.");
54             break;
55         }
56
57         Patient patient1 = new Patient(
58             id,
59             name,
60             mobile,
61             department,
62             doctor
63         );
64
65         patients.add(patient1);
66         patientMap.put(id, patient1);
67
68         System.out.println("Patient Registered Successfully.");
69
70         break;
71
72     case 2:
73
74         if (!sc.hasNextLine()) return;
75
76         String searchId = sc.nextLine().trim();
77
78         if (patientMap.containsKey(searchId)) {
79
80             Patient patient2 =
81                 patientMap.get(searchId);
82

```



```
85         System.out.println("ID: " + patient2.id);
86         System.out.println("Name: " + patient2.name);
87         System.out.println("Mobile: " + patient2.mobile);
88         System.out.println(
89             "Department: " +
90             patient2.department
91         );
92         System.out.println(
93             "Doctor: " +
94             patient2.doctor
95         );
96     } else {
97     }
98     System.out.println("Patient Not Found");
99 }
100 }
101 break;
102
103 case 3:
104
105     System.out.println("ID Name Department");
106
107     for (Patient patient3 : patients) {
108
109         System.out.println(
110             patient3.id + " " +
111             patient3.name + " " +
112             patient3.department
113         );
114     }
115     break;
116
117 case 4:
118
119     HashMap<String, Integer> departmentCount =
120         new HashMap<>();
121
122     for (Patient patient4 : patients) {
123
124         String dept = patient4.department;
125
126         if (departmentCount.containsKey(dept)) {
127
128             departmentCount.put(
129                 dept,
130                 departmentCount.get(dept) + 1
131             );
132         } else {
133
134             departmentCount.put(dept, 1);
135         }
136     }
137
138     System.out.println("Department Count");
139
140     HashSet<String> printed =
141         new HashSet<>();
142
143     for (Patient patient5 : patients) {
144
145         String dept = patient5.department;
146
147         if (!printed.contains(dept)) {
148
149             System.out.println(
150                 dept + ": " +
151                 departmentCount.get(dept)
152             );
153
154             printed.add(dept);
155         }
156     }
157     break;
158
159 case 5:
160
161     sc.close();
162     return;
163 }
```



```
171
172         default:
173
174             break;
175     }
176 }
177
178 sc.close();
179}
180
181
182 static class Patient {
183
184     String id;
185     String name;
186     String mobile;
187     String department;
188     String doctor;
189
190     Patient(String id, String name, String mobile,
191             String department, String doctor) {
192
193         this.id = id;
194         this.name = name;
195         this.mobile = mobile;
196         this.department = department;
197         this.doctor = doctor;
198     }
199 }
200
201
202 static String properCase(String name) {
203
204     name = name.toLowerCase().trim();
205
206     String result = "";
207     boolean newWord = true;
208
209     for (int i = 0; i < name.length(); i++) {
210
211         char ch = name.charAt(i);
212
213         if (ch == ' ') {
214
215             result = result + ch;
216             newWord = true;
217
218         } else {
219
220             if (newWord) {
221
222                 result = result +
223                     Character.toUpperCase(ch);
224
225                 newWord = false;
226
227             } else {
228
229                 result = result + ch;
230             }
231         }
232     }
233
234     return result.trim();
235 }
236
237
238 static boolean validName(String name) {
239
240     if (name.length() == 0) {
241         return false;
242     }
243
244     for (int i = 0; i < name.length(); i++) {
245
246         char ch = name.charAt(i);
247
248         if (!Character.isLetter(ch) && ch != ' ') {
249             return false;
250         }
251     }
252
253     return true;
254 }
255
```

```

213         if (ch == ' ') {
214
215             result = result + ch;
216             newWord = true;
217
218         } else {
219
220             if (newWord) {
221
222                 result = result +
223                     Character.toUpperCase(ch);
224
225                 newWord = false;
226
227             } else {
228
229                 result = result + ch;
230             }
231         }
232     }
233
234     return result.trim();
235 }
236
237
238 static boolean validName(String name) {
239
240     if (name.length() == 0) {
241         return false;
242     }
243
244     for (int i = 0; i < name.length(); i++) {
245
246         char ch = name.charAt(i);
247
248         if (!Character.isLetter(ch) && ch != ' ') {
249             return false;
250         }
251     }
252
253     return true;
254 }
255
256
257 static boolean validMobile(String mobile) {
258
259     if (mobile.length() != 10) {
260         return false;
261     }
262
263     for (int i = 0; i < mobile.length(); i++) {
264
265         if (!Character.isDigit(mobile.charAt(i))) {
266             return false;
267         }
268     }
269
270     return true;
271 }
272

```

Input

```

1
P101
Rahul Kumar
9876543210

```

Output

```

Patient Registered Successfully.
Patient Registered Successfully.
ID Name Department
P101 Rahul Kumar CARDIOLOGY
P102 Priya ORTHO
Department Count

```

Visible Test Cases

Scenario

Online Library Book Management System

A university library is planning to automate its book management process. The librarian wants a Java application that stores book details, prevents duplicate book entries, allows students to search for books quickly, and generates category-wise reports.

The application should use Java String handling and Java Collection Framework to efficiently manage the library records.

Requirements

Each book contains the following details:

❏ Book ID

❏ Book Title

❏ Author Name

❏ Category

❏ Publisher

Use:

❏ ArrayList to store all book objects.

❏ HashMap to store Book ID as the key for quick searching.

❏ HashSet to maintain unique author names.

Menu Driven Program

1. Add Book
2. Search Book by ID
3. Display All Books
4. Display Unique Authors
5. Category-wise Book Count
6. Exit

Requirement 1 – Add Book (4 Marks)

Accept

Book ID

Book Title

Author Name

Category

Publisher

Validate the following:

Book ID must be unique.

Remove leading and trailing spaces.

Book Title should be stored in Title Case.

Category should be converted to uppercase.

Publisher name should not be empty.

Store the record in both

ArrayList

HashMap

Requirement 2 – Search Book (3 Marks)

Accept Book ID.

If found, display

Book ID

Book Title

Author

Category

Publisher

Otherwise,

Book Not Available

Requirement 3 – Display All Books (4 Marks)

Display all records in tabular format.

Example

ID	TITLE	AUTHOR	CATEGORY
----	-------	--------	----------

B101	Java Programming	Herbert Schildt	PROGRAMMING
------	------------------	-----------------	-------------

B102	Data Structures	Mark Allen	COMPUTER SCIENCE
------	-----------------	------------	------------------

Requirement 4 – Display Unique Authors (3 Marks)

Display all distinct author names using HashSet.

Example

Unique Authors

Herbert Schildt

James Gosling

Yashavant Kanetkar

Mark Allen

Requirement 5 – Category-wise Book Count (4 Marks)

Display the number of books available in each category using
HashMap<String,Integer>

Mark Allen

Requirement 5 – Category-wise Book Count (4 Marks)

Display the number of books available in each category using

HashMap<String,Integer>

Example

PROGRAMMING : 12

DATABASE : 8

NETWORKING : 6

Requirement 6 – String Operations (2 Marks)

For every book,

Display

Length of Book Title

Number of words in the title

Category in uppercase

Example

Input

Java Programming Language

Output

Length : 25

Words : 3

Category : PROGRAMMING

Sample Input

1

B101

Java Programming

Herbert Schildt

Programming

McGraw Hill

1

B102

Data Structures

Mark Allen

Computer Science

Pearson

3

4

5

2

B101

6

Sample Output

Book Added Successfully.

Book Added Successfully.

Book ID Title Author

B101 Java Programming Herbert Schildt

B102 Data Structures Mark Allen

Unique Authors

Herbert Schildt

Mark Allen

Category-wise Count

PROGRAMMING : 1

COMPUTER SCIENCE : 1

Book Details

Book ID : B101

Title : Java Programming

Author : Herbert Schildt

Category : PROGRAMMING

Publisher : McGraw Hill

```

1 import java.util.*;
2
3 public class Main {
4
5     static ArrayList<Book> books = new ArrayList<>();
6     static HashMap<String, Book> bookMap = new HashMap<>();
7     static HashSet<String> authors = new HashSet<>();
8
9     public static void main(String[] args) {
10
11         Scanner sc = new Scanner(System.in);
12
13         while (true) {
14
15             int choice = Integer.parseInt(sc.nextLine());
16
17             switch (choice) {
18
19                 case 1:
20
21                     String id = sc.nextLine().trim();
22
23                     if (bookMap.containsKey(id)) {
24                         System.out.println("Duplicate Book ID");
25                         break;
26                     }
27
28                     String title = sc.nextLine().trim();
29                     String author = sc.nextLine().trim();
30                     String category = sc.nextLine().trim();
31                     String publisher = sc.nextLine().trim();
32
33                     if (publisher.isEmpty()) {
34                         System.out.println("Publisher name should not be empty");
35                         break;
36                     }
37
38                     Book book = new Book(
39                         id,
40                         title,
41                         author,
42                         category,
43                         publisher
44                     );
45
46                     books.add(book);
47                     bookMap.put(id, book);
48                     authors.add(book.author);
49
50                     System.out.println("Book Added Successfully.\n");
51                     break;
52
53                 case 2:
54
55                     String searchId = sc.nextLine().trim();
56
57                     if (bookMap.containsKey(searchId)) {
58
59                         Book b = bookMap.get(searchId);
60
61                         System.out.println("Book Details");
62                         System.out.println("Book ID: " + b.id);
63                         System.out.println("Title: " + b.title);
64                         System.out.println("Author: " + b.author);
65                         System.out.println("Category: " + b.category);
66                         System.out.println("Publisher: " + b.publisher);
67
68                     } else {
69
70                         System.out.println("Book Not Available");
71                     }
72
73                     break;
74
75                 case 3:
76
77                     System.out.println("-----");
78
79                     break;
80
81                 default:
82                     System.out.println("Invalid Choice");
83                     break;
84             }
85         }
86     }
87 }

```



```

83         for (Book b : books) {
84
85             System.out.println(
86                 b.id + " * " +
87                 b.title + " * " +
88                 b.author
89             );
90         }
91
92         break;
93
94
95     case 4:
96
97         System.out.println("Unique Authors");
98
99         for (String a : authors) {
100             System.out.println(a);
101         }
102
103         break;
104
105
106     case 5:
107
108         System.out.println("Category-wise Count");
109
110         HashMap<String, Integer> count =
111             new HashMap<>();
112
113         for (Book b : books) {
114
115             if (count.containsKey(b.category)) {
116
117                 int oldCount = count.get(b.category);
118                 count.put(b.category, oldCount + 1);
119
120             } else {
121
122                 count.put(b.category, 1);
123             }
124         }
125
126         for (Map.Entry<String, Integer> entry :
127             count.entrySet()) {
128
129             System.out.println(
130                 entry.getKey() + " : " +
131                 entry.getValue()
132             );
133         }
134
135         break;
136
137
138     default:
139
140         return;
141     }
142 }
143 }
144
145
146 static class Book {
147
148     String id;
149     String title;
150     String author;
151     String category;
152     String publisher;
153
154     Book(String id,
155          String title,
156          String author,
157          String category,
158          String publisher) {
159
160         this.id = id.trim();
161         this.title = toTitleCase(title.trim());
162         this.author = author.trim();
163         this.category = category.trim().toUpperCase();
164         this.publisher = publisher.trim();
165     }
166 }
167
168
169 static String toTitleCase(String str) {
170
171     String result = "";
172     boolean newWord = true;
173
174     for (int i = 0; i < str.length(); i++) {
175

```

```

143     }
144
145
146     static class Book {
147
148         String id;
149         String title;
150         String author;
151         String category;
152         String publisher;
153
154         Book(String id,
155             String title,
156             String author,
157             String category,
158             String publisher) {
159
160             this.id = id.trim();
161             this.title = toTitleCase(title.trim());
162             this.author = author.trim();
163             this.category = category.trim().toUpperCase();
164             this.publisher = publisher.trim();
165         }
166     }
167
168
169     static String toTitleCase(String str) {
170
171         String result = "";
172         boolean newWord = true;
173
174         for (int i = 0; i < str.length(); i++) {
175
176             char ch = str.charAt(i);
177
178             if (Character.isWhitespace(ch)) {
179
180                 result = result + ch;
181                 newWord = true;
182
183             } else {
184
185                 if (newWord) {
186
187                     result = result +
188                         Character.toUpperCase(ch);
189
190                     newWord = false;
191
192                 } else {
193
194                     result = result +
195                         Character.toLowerCase(ch);
196                 }
197             }
198         }
199
200         return result.trim();
201     }
202 }

```

Input

```

1
B101
Java Programming
Herbert Schildt

```

Output

```

Book Added Successfully.
Book Added Successfully.
-----
Book ID Title Author

```

Visible Test Cases

Test Case 1

Test Case 2